

3.2 · Discussion — The Cost of Dynamic Reasoning

Week 2 · Wednesday Jul 22 — Measurement, Evaluation, and the Startup Pitch

Generative AI & Sustainability · Munich Summer School 2026



CAL POLY



Today's paper

The Cost of Dynamic Reasoning: Demystifying AI Agents and Test-Time Scaling from an AI Infrastructure Perspective

Kim, Shin, Chung, Rhu — IEEE HPCA 2026 · arXiv:2506.04301

Yesterday's paper measured what it costs to **create** a model. This one measures what it costs to **run agents** — loops, tools, retries — which is what you designed on Monday.

Feel free to reference your **admission ticket**.

What they measured

Five agent designs — **CoT · ReAct · Reflexion · LATS · LLMCompiler** — run end to end on real hardware:

- Benchmarks: **HotpotQA, WebShop, MATH, HumanEval**
- Models: Llama **8B** (1× A100 40GB) and **70B** (8× A100), served with vLLM
- Baseline for comparison: an ordinary **single-turn chat query** (ShareGPT)
- Reported per **completed query**, not per token — because the cost comes from *loops*, not model size

Notable image: five loop structures

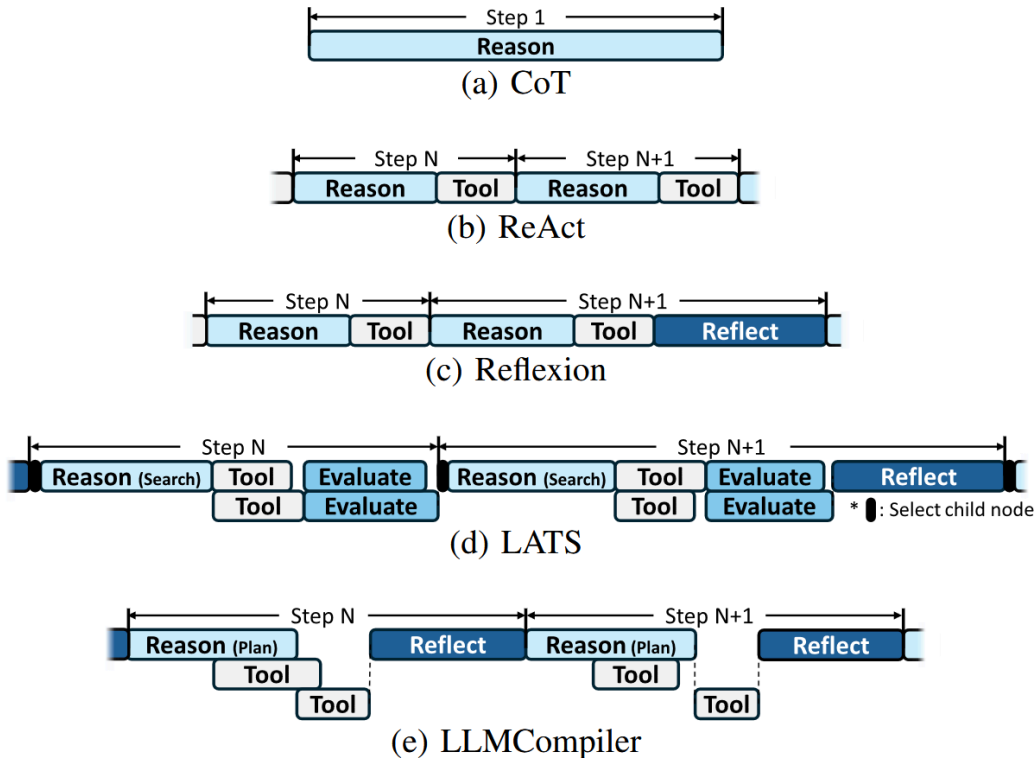


Fig. 3. Execution timeline of each AI agent.

- **Pattern:** CoT is one box; every other design repeats **Reason** → **Tool** — the loop is where the cost lives, not the model
- **Depth:** Reflexion adds a *Reflect* per step; LATS branches into parallel *Tool/Evaluate* pairs — **71 LLM calls per request**
- **Overlap:** only LLMCompiler runs tools *during* reasoning, and recovers just **18.2%** of latency

What it costs

Workload	8B	70B
Single-turn chat (baseline)	0.32 Wh	2.55 Wh
LATS	22.76 Wh (71.7×)	158.48 Wh (62.1×)
Reflexion	41.53 Wh (130.9×)	348.41 Wh (136.5×)

- That **136.5×** is the multiplier you used in your Week 1 compute assignment — this is where it came from
- Where the time goes: LLM inference **69.4%** · tool execution **30.2%**
- On Wikipedia-API workloads the GPU sits **idle up to 54.5%** of execution time — waiting on tools, still powered, still blocking others

Test-time scaling: the returns curve

- **Sequential** (more reflection rounds): the *same* ~4% accuracy gain cost 16.9 → 25.6 s early on, and 56.0 → 269.5 s later — roughly **31× more** for the same improvement
- **Parallel** (more candidates at once): 1 → 16 children bought **+14.4 accuracy points** and **−196 s** average latency — but with more concurrency and memory pressure
- A **small model with test-time scaling** can approach a large model's accuracy at lower total energy

The lesson is not "less compute." It is **compute-aware design**: where you spend it matters more than how much.

Scaled up

If an agentic service were served at today's chatbot traffic:

- Reflexion-70B at **71.4M queries/day** $\approx$ **1 GW** · **24.89 GWh/day** — about the daily electricity of **Seattle**
- At **Google-Search scale** $\approx$ **200 GW** — roughly half the average U.S. grid load

The authors' answer is not abstinence: **mix small and large models, scale adaptively, cap loops, quantize and distill.**

Why we discuss instead of lecture

Tomorrow an audience will ask what your system costs to run — not to train.

Today you practice arguing that with evidence, and listening the way that audience will listen to you.

Fishbowl

Some of you inside the circle. Some outside, listening with a role.

We swap at halftime — everyone does both jobs.

Fishbowl rules + observer roles

Inner circle: discuss the question; build on, question, or challenge each other. Use your admission ticket. No hands — it's a conversation.

Observers — each takes one listening role:

Role	Listen for
Evidence tracker	Claims backed by numbers from the paper vs. vibes
Connector	Links to yesterday's paper or our course concepts
Devil's advocate	The strongest point <i>nobody</i> made
Balance monitor	Who spoke, who was cut off, what invited others in
Summarizer	The 2–3 ideas the group converged on

Round 1 question

"A Reflexion agent spends $\sim 130\times$ the energy of a chatbot answer — and the last reflection rounds buy almost nothing. When is an agent worth $100\times$ the energy of a plain answer? Name a task where it is, one where it isn't, and the threshold you'd defend."

Observers: hold your report-out to **one sentence per role**.

Round 2 question

"On the Wikipedia-API workload, the GPU sits idle up to 54% of the time — burning money and blocking others — while the agent waits for tools. Your agents call sensors, calendars, and booking APIs. What would an honest 'energy label' for an *agentic* product report?"

Swap circles first. New observers, same roles.

Whole-group synthesis — back to your startups

1. Take out your **Monday agent map**. Trace one user request through it and count the LLM calls and tool waits. What is *your* multiplier, honestly estimated?
2. Which single change cuts the most energy while losing the least — **capping loop iterations, caching, routing to a small model, or parallelizing tool calls**? Commit to one; it goes in this afternoon's evaluation plan.
3. The paper's projections assume everyone deploys maximal agents. Your pitch can say: *we designed against that default, and here is the number that proves it*. **What is that number?**

Close

- Summarizer observers: your convergence points, one per round
- **Hand in your admission tickets**
- Next (M3): the pitch — stakeholders, framing, and a live look at your own decks
- This afternoon's mini-lab produces **one measured row** of that energy label